

Série d'exercices B

Fonctions et fonctions récursives

1. Ecrivez une fonction récursive qui prend une `List[Int]` en paramètre et qui somme tous les éléments de la liste, sans utiliser la fonction définie `List.sum`.

2. Définissez un `Vector[Int]` contenant les valeurs de 1 à 1000.

Ecrivez une fonction qui prend un `Int` en paramètre.

- Si cet `Int` est un multiple de trois, retourne son cube.
- Si cet `Int` est un multiple de deux, retourne son carré.
- Si cet `Int` est impair, retourne -1.

Appliquez cette fonction à toutes les valeurs du vecteur.

3. Définissez un `Vector[Int]` contenant les valeurs de 1 à 10.

Ecrivez une fonction qui prend un `Int` en paramètre, et retourne le factoriel de ce nombre, en utilisant une implémentation récursive.

Ecrivez une fonction qui prend un `Int` en paramètre, et retourne le nombre de fibonacci correspondant, en utilisant une implémentation récursive. Ecrivez la même fonction en utilisant une implémentation récursive terminale.

Appliquez les fonctions au vecteur comme suit :

- Si le nombre est pair, lui appliquer la fonction factoriel.
- Si le nombre est impair, lui appliquer la fonction fibonacci.

4. L'exponentiation rapide est un algorithme utilisé pour calculer rapidement de grandes puissances entières :

$$b^{2n} = (b^2)^n = (b^n)^2, \quad b^{2n+1} = b \cdot b^{2n}$$

Écrire une fonction récursive terminale `fastExp` qui implémente l'exponentiation rapide.

```
def fastExp(base: Int, exp: Int): Int = ???
```